Purbanchal University

Faculty of Science & Technology

A LAB REPORT ON

MACHINE LEARNING

Submitted to

Department of Computer Application

Gomendra Multiple College

In partial fulfillment of the requirements for the Masters of Computer Application

Submitted by

Achyut Chapagain (Roll No: 01)

Regd No: 004-3-3-01635-2025

Under the Supervision of

Mr. Madan Tiwari

Table of Contents

1. Basic list, tuple, and dictionary operations.

Title	Basic list, tuple, and dictionary operations.
Objective	To demonstrate basic operations on list, tuple, and dictionary using Python.
Required Tools/Software	Python 3.x, VS Code/Jupyter Notebook

Problem Statement

To demonstrate basic operations on list, tuple, and dictionary using Python.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
# List operations
numbers = [10, 20, 30]
numbers.append(40)
numbers.remove(20)
print('List:', numbers)
print('First element:', numbers[0])
# Tuple operations
marks = (80, 75, 90, 85)
print('Tuple:', marks)
print('Maximum marks:', max(marks))
print('Length of tuple:', len(marks))
# Dictionary operations
student = {'name': 'Achyut', 'age': 24, 'course': 'MCA'}
student['semester'] = 2
student['age'] = 24
print('Dictionary:', student)
print('Student name:', student['name'])
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python programs/program1.py
List: [10, 30, 40]
First element: 10
Tuple: (80, 75, 90, 85)
Maximum marks: 90
Length of tuple: 4
Dictionary: {'name': 'Achyut', 'age': 24, 'course': 'MCA', 'semester': 2}
Student name: Achyut
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Lists, tuples, and dictionaries are important Python data structures used to store and process data in AI and ML programs.

2. Create a 5*5 matrix using NumPy and compute transpose

Title	Create a 5x5 matrix using NumPy and compute transpose
Objective	To create a 5x5 matrix and find its transpose using NumPy.
Required Tools/Software	Python 3.x, NumPy

Problem Statement

To create a 5x5 matrix and find its transpose using NumPy.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import numpy as np
matrix = np.arange(1, 26).reshape(5, 5)
transpose = matrix.T
print('Original Matrix:')
print(matrix)
print('Transpose Matrix:')
print(transpose)
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python programs/program2.py
Original Matrix:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
Transpose Matrix:
[[ 1  6 11 16 21]
 [ 2  7 12 17 22]
 [ 3  8 13 18 23]
 [ 4  9 14 19 24]
 [ 5 10 15 20 25]]
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

NumPy makes matrix operations simple and efficient for scientific and machine learning tasks.

3. Data exploration and preprocessing

Title	Data exploration and preprocessing
Objective	To perform basic data exploration and preprocessing on a sample dataset.
Required Tools/Software	Python 3.x, Pandas, NumPy

Problem Statement

To perform basic data exploration and preprocessing on a sample dataset.

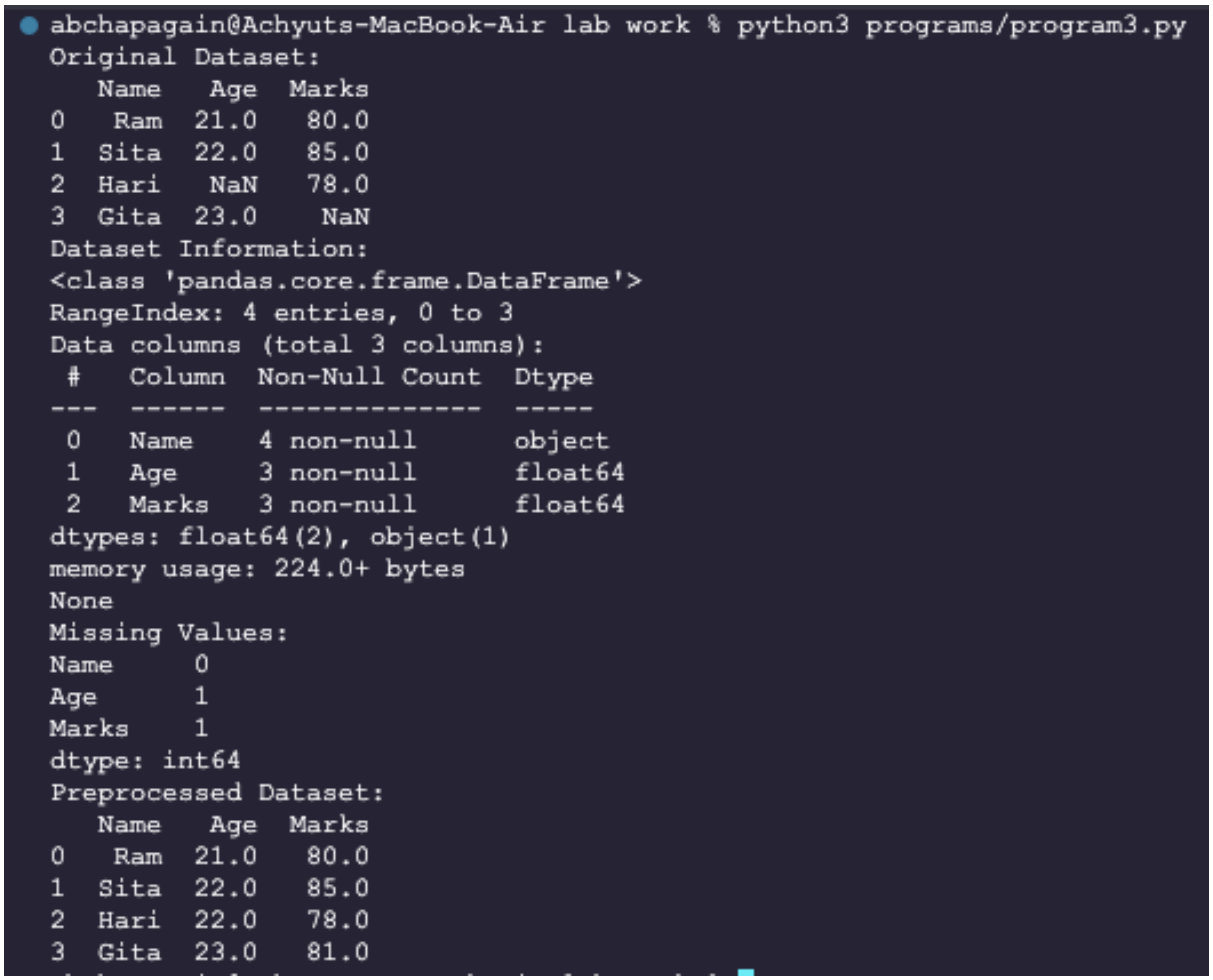
Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import pandas as pd
# Sample dataset
data = {
'Name': ['Ram', 'Sita', 'Hari', 'Gita'],
'Age': [21, 22, None, 23],
'Marks': [80, 85, 78, None]
}
df = pd.DataFrame(data)
print('Original Dataset:')
print(df)
print('Dataset Information:')
print(df.info())
print('Missing Values:')
print(df.isnull().sum())
# Fill missing values using mean
numeric_cols = df.select_dtypes(include='number').columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
print('Preprocessed Dataset:')
print(df)
```

Expected Output



```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program3.py
Original Dataset:
    Name   Age  Marks
0    Ram  21.0   80.0
1   Sita  22.0   85.0
2   Hari   NaN   78.0
3   Gita  23.0    NaN
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    4 non-null      object
 1   Age     3 non-null      float64
 2   Marks   3 non-null      float64
dtypes: float64(2), object(1)
memory usage: 224.0+ bytes
None
Missing Values:
Name     0
Age      1
Marks    1
dtype: int64
Preprocessed Dataset:
    Name   Age  Marks
0    Ram  21.0   80.0
1   Sita  22.0   85.0
2   Hari  22.0   78.0
3   Gita  23.0   81.0
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Exploration helps understand data quality, and preprocessing prepares data for ML algorithms.

4. Load a CSV dataset and display summary statistics

Title	Load a CSV dataset and display summary statistics
Objective	To load a CSV dataset using Pandas and display its basic statistical summary.
Required Tools/Software	Python 3.x, Pandas, CSV dataset

Problem Statement

To load a CSV dataset using Pandas and display its basic statistical summary.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import pandas as pd
# students.csv is present in the same folder
# Columns: Hours, Marks, Attendance
df = pd.read_csv('students.csv')
print('First five records:')
print(df.head())
print('Summary Statistics:')
print(df.describe())
print('Dataset Shape:', df.shape)
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program4.py
First five records:
   Hours  Marks  Attendance
0      2     45          70
1      3     52          75
2      4     60          80
3      5     66          82
4      6     72          85
Summary Statistics:
           Hours      Marks  Attendance
count  10.00000  10.000000   10.000000
mean    6.50000  73.300000   85.500000
std     3.02765  17.333654    8.872554
min     2.00000  45.000000   70.000000
25%     4.25000  61.500000   80.500000
50%     6.50000  75.000000   86.500000
75%     8.75000  87.000000   91.500000
max    11.00000  96.000000   98.000000
Dataset Shape: (10, 3)
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Pandas is useful for loading datasets and quickly understanding numerical columns using summary statistics.

5. Handle missing values using mean/median

Title	Handle missing values using mean/median
Objective	To replace missing values in a dataset using mean and median values.
Required Tools/Software	Python 3.x, Pandas, NumPy

Problem Statement

To replace missing values in a dataset using mean and median values.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import pandas as pd
import numpy as np
data = {
'Age': [21, 22, np.nan, 24, 25],
'Salary': [20000, np.nan, 25000, 30000, 28000]
}
df = pd.DataFrame(data)
print('Before Handling Missing Values:')
print(df)
# Replace missing Age with mean and Salary with median
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].median())
print('After Handling Missing Values:')
print(df)
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program5.py
Before Handling Missing Values:
    Age    Salary
0  21.0  20000.0
1  22.0      NaN
2   NaN  25000.0
3  24.0  30000.0
4  25.0  28000.0
After Handling Missing Values:
    Age    Salary
0  21.0  20000.0
1  22.0  26500.0
2  23.0  25000.0
3  24.0  30000.0
4  25.0  28000.0
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Missing value handling improves dataset completeness and prevents errors during model training.

6. Simple linear regression for house price prediction

Title	Simple linear regression for house price prediction
Objective	To predict house prices based on square footage using simple linear regression.
Required Tools/Software	Python 3.x, NumPy, Pandas, Scikit-learn

Problem Statement

To predict house prices based on square footage using simple linear regression.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import pandas as pd
from sklearn.linear_model import LinearRegression
X = [[800], [1000], [1200], [1500], [1800]]
y = [150000, 180000, 210000, 260000, 300000]
model = LinearRegression()
model.fit(X, y)
area = [[1400]]
prediction = model.predict(area)
print('Slope:', model.coef_[0])
print('Intercept:', model.intercept_)
print('Predicted price for 1400 sq.ft:', prediction[0])
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program6.py
Slope: 151.89873417721518
Intercept: 28607.59493670889
Predicted price for 1400 sq.ft: 241265.82278481012
```

Observation

The model successfully predicts house price based on square footage. The result shows a positive relationship between area and price.

Conclusion

Linear regression learns the relationship between square footage and price and can predict values for new data.

7. Plot regression line and interpret slope/intercept

Title	Plot regression line and interpret slope/intercept
Objective	To plot a regression line using Matplotlib and interpret slope and intercept.
Required Tools/Software	Python 3.x, Matplotlib, Scikit-learn

Problem Statement

To plot a regression line using Matplotlib and interpret slope and intercept.

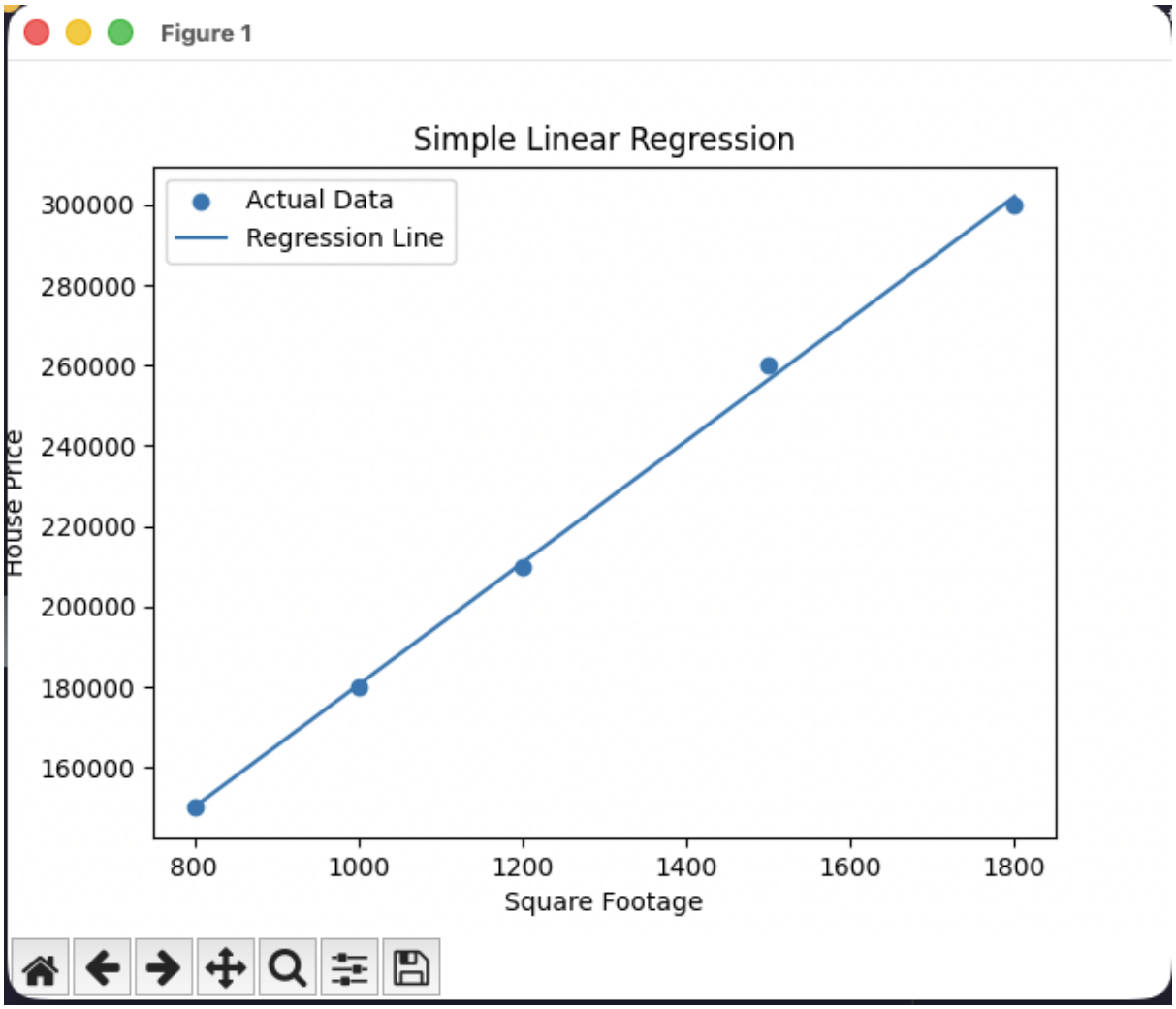
Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
X = [[800], [1000], [1200], [1500], [1800]]
y = [150000, 180000, 210000, 260000, 300000]
model = LinearRegression()
model.fit(X, y)
predicted = model.predict(X)
plt.scatter(X, y, label='Actual Data')
plt.plot(X, predicted, label='Regression Line')
plt.xlabel('Square Footage')
plt.ylabel('House Price')
plt.title('Simple Linear Regression')
plt.legend()
plt.show()
print('Slope:', model.coef_[0])
print('Intercept:', model.intercept_)
```

Expected Output



Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

The slope shows price increase per square foot, and the intercept shows the predicted value when input is zero.

8. Naive Bayes classifier for spam detection

Title	Naive Bayes classifier for spam detection
Objective	To classify emails as spam or non-spam using Naive Bayes classifier.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To classify emails as spam or non-spam using Naive Bayes classifier.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
emails = ['win money now', 'hello friend', 'claim free prize', 'meeting at 5 pm']
labels = ['spam', 'ham', 'spam', 'ham']
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)
model = MultinomialNB()
model.fit(X, labels)
test_email = ['free money prize']
test_vector = vectorizer.transform(test_email)
print('Prediction:', model.predict(test_vector)[0])
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program8.py
Prediction: spam
abchapagain@Achyuts-MacBook-Air lab work %
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Naive Bayes works well for text classification because it uses word probabilities to make predictions.

9. Evaluate Naive Bayes using confusion matrix and classification report

Title	Evaluate Naive Bayes using confusion matrix and classification report
Objective	To evaluate a classifier using accuracy, confusion matrix, and classification report.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To evaluate a classifier using accuracy, confusion matrix, and classification report.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
y_true = ['spam', 'ham', 'spam', 'ham', 'spam']
y_pred = ['spam', 'ham', 'spam', 'spam', 'spam']
print('Accuracy:', accuracy_score(y_true, y_pred))
print('Confusion Matrix:')
print(confusion_matrix(y_true, y_pred, labels=['spam', 'ham']))
print('Classification Report:')
print(classification_report(y_true, y_pred))
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program9.py
Accuracy: 0.8
Confusion Matrix:
[[3 0]
 [1 1]]
Classification Report:
              precision    recall  f1-score   support

         ham       1.00      0.50      0.67         2
        spam       0.75      1.00      0.86         3

    accuracy                           0.80         5
   macro avg       0.88      0.75      0.76         5
weighted avg       0.85      0.80      0.78         5
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Naive Bayes works well for text classification because it uses word probabilities to make predictions.

10. Logistic regression for student pass prediction

Title	Logistic regression for student pass prediction
Objective	To predict whether a student passes based on study hours using logistic regression.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To predict whether a student passes based on study hours using logistic regression.

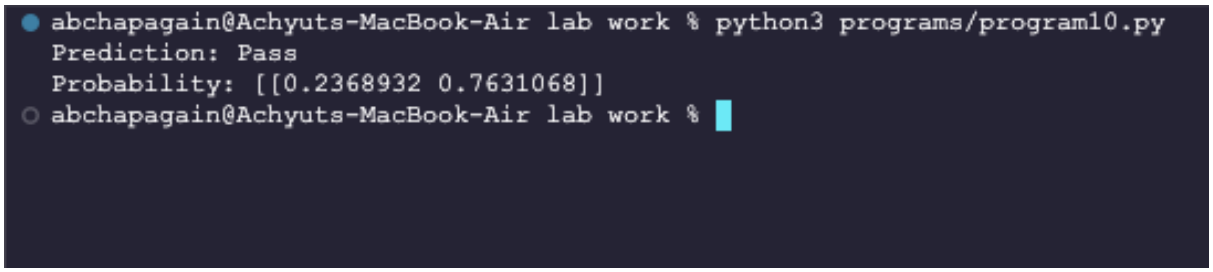
Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.linear_model import LogisticRegression
X = [[1], [2], [3], [4], [5], [6], [7], [8]]
y = [0, 0, 0, 0, 1, 1, 1, 1] # 0 = Fail, 1 = Pass
model = LogisticRegression()
model.fit(X, y)
hours = [[5.5]]
result = model.predict(hours)
probability = model.predict_proba(hours)
print('Prediction:', 'Pass' if result[0] == 1 else 'Fail')
print('Probability:', probability)
```

Expected Output



Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Logistic regression is useful for binary classification problems. In this experiment, it predicts whether a student will pass or fail based on study hours.

11. Visualize logistic regression decision boundary

Title	Visualize logistic regression decision boundary
Objective	To visualize the decision boundary of logistic regression using Matplotlib.
Required Tools/Software	Python 3.x, NumPy, Matplotlib, Scikit-learn

Problem Statement

To visualize the decision boundary of logistic regression using Matplotlib.

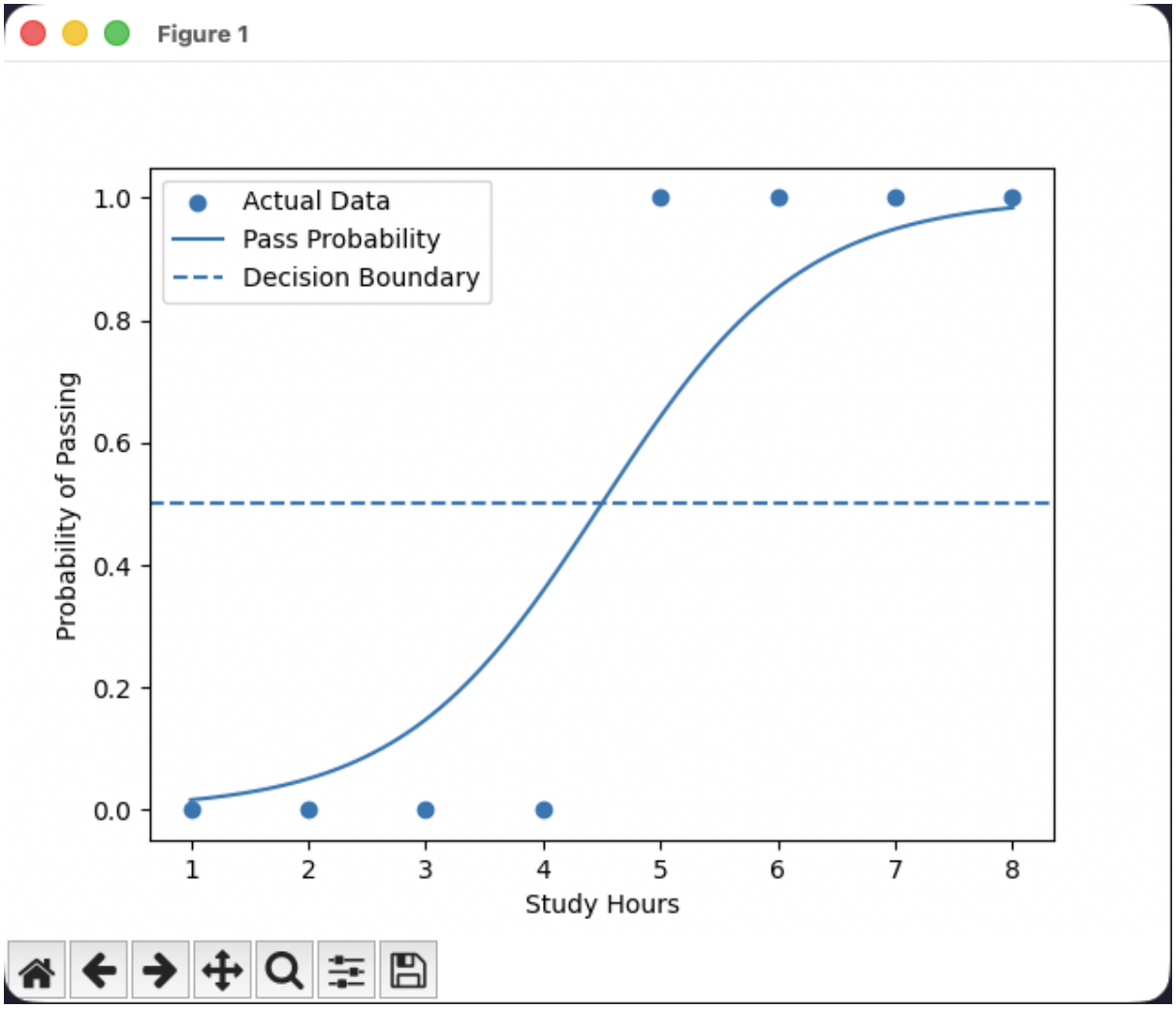
Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
model = LogisticRegression()
model.fit(X, y)
x_range = np.linspace(1, 8, 100).reshape(-1, 1)
y_prob = model.predict_proba(x_range)[:, 1]
plt.scatter(X, y, label='Actual Data')
plt.plot(x_range, y_prob, label='Pass Probability')
plt.axhline(0.5, linestyle='--', label='Decision Boundary')
plt.xlabel('Study Hours')
plt.ylabel('Probability of Passing')
plt.legend()
plt.show()
```

Expected Output



Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

The graph shows how probability changes with study hours and where the model separates pass and fail classes.

12. Decision tree classifier for Iris dataset

Title	Decision tree classifier for Iris dataset
Objective	To build a decision tree classifier using the Iris dataset.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To build a decision tree classifier using the Iris dataset.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python
3 programs/program12.py
Accuracy: 1.0
abchapagain@Achyuts-MacBook-Air lab work %
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Decision trees classify data by splitting it into branches based on feature values.

13. Visualize decision tree structure

Title	Visualize decision tree structure
Objective	To visualize a decision tree using sklearn.tree.plot_tree.
Required Tools/Software	Python 3.x, Matplotlib, Scikit-learn

Problem Statement

To visualize a decision tree using sklearn.tree.plot_tree.

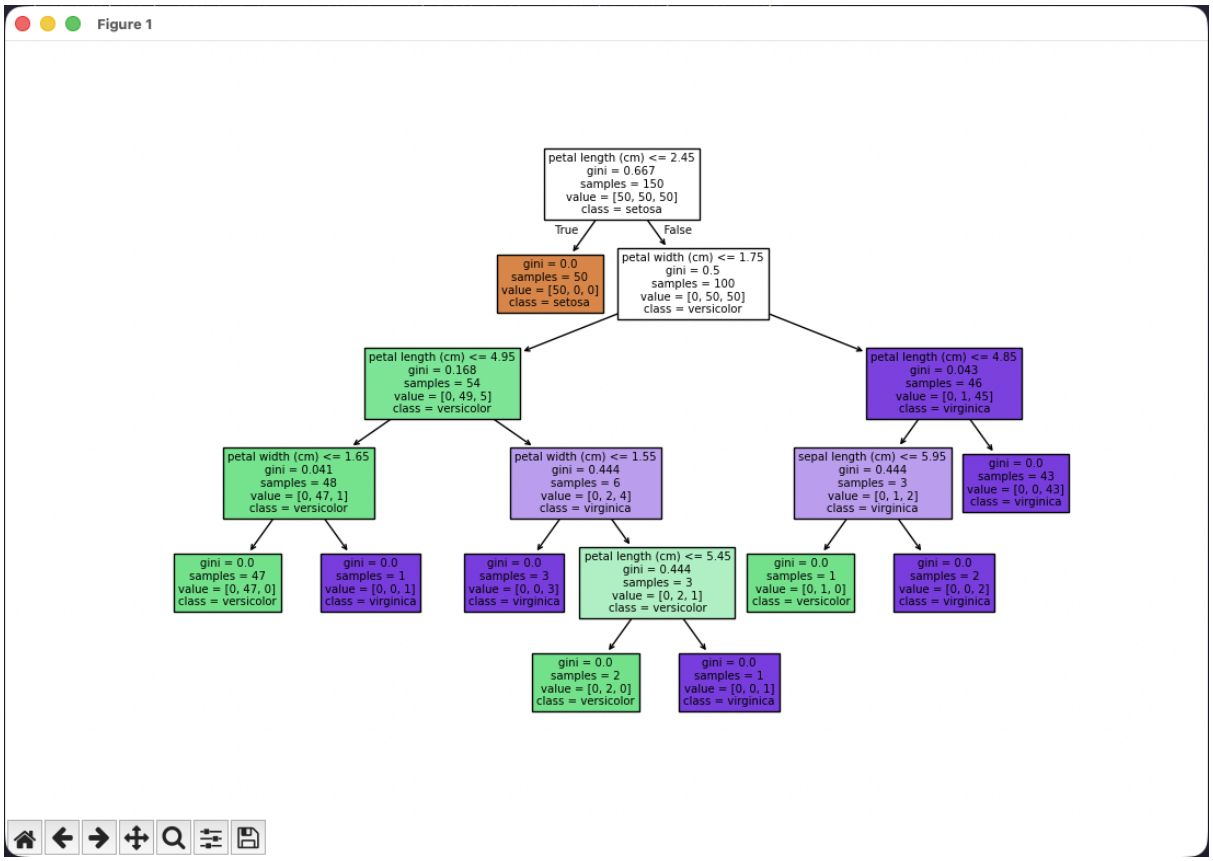
Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
iris = load_iris()
model = DecisionTreeClassifier(random_state=42)
model.fit(iris.data, iris.target)
plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
plt.show()
```

Expected Output



Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Tree visualization helps understand how the decision tree makes classification decisions.

14. SVM with linear kernel on Iris dataset

Title	SVM with linear kernel on Iris dataset
Objective	To implement Support Vector Machine with a linear kernel on the Iris dataset.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To implement Support Vector Machine with a linear kernel on the Iris dataset.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.3, random_state=1)
model = SVC(kernel='linear')
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program14.py
Accuracy: 1.0
abchapagain@Achyuts-MacBook-Air lab work %
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

SVM with a linear kernel separates classes using an optimal separating hyperplane.

15. K-NN classifier for handwritten digits

Title	K-NN classifier for handwritten digits
Objective	To classify handwritten digits using K-Nearest Neighbors on the digits dataset.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To classify handwritten digits using K-Nearest Neighbors on the digits dataset.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.25, random_state=42)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, pred))
print('First predicted digit:', pred[0])
```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program15.py
Accuracy: 0.9866666666666667
First predicted digit: 6
abchapagain@Achyuts-MacBook-Air lab work %
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

K-NN classifies a sample based on the majority class among its nearest neighbors.

16. Experiment with different K values in K-NN

Title	Experiment with different K values in K-NN
Objective	To observe accuracy changes by using different values of K in K-NN.
Required Tools/Software	Python 3.x, Scikit-learn

Problem Statement

To observe accuracy changes by using different values of K in K-NN.

Procedure

1. Import the required Python libraries.
2. Prepare or load the required dataset.
3. Apply the machine learning/data processing technique.
4. Display the result and evaluate the output.
5. Write observation and conclusion.

Program

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.25, random_state=42)
for k in [1, 3, 5, 7, 9]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print('K =', k, 'Accuracy =', accuracy_score(y_test, pred))

```

Expected Output

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program16.py
K = 1 Accuracy = 0.9822222222222222
K = 3 Accuracy = 0.9866666666666667
K = 5 Accuracy = 0.9933333333333333
K = 7 Accuracy = 0.9933333333333333
K = 9 Accuracy = 0.9866666666666667
abchapagain@Achyuts-MacBook-Air lab work %
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Changing K affects model performance, so the best value should be selected by experimentation.

17. WEKA K-means clustering on a dataset

Title	WEKA K-means clustering on a dataset
Objective	To perform K-means clustering in WEKA on a selected dataset.
Required Tools/Software	WEKA, sample ARFF/CSV dataset

Problem Statement

To perform K-means clustering in WEKA on a selected dataset.

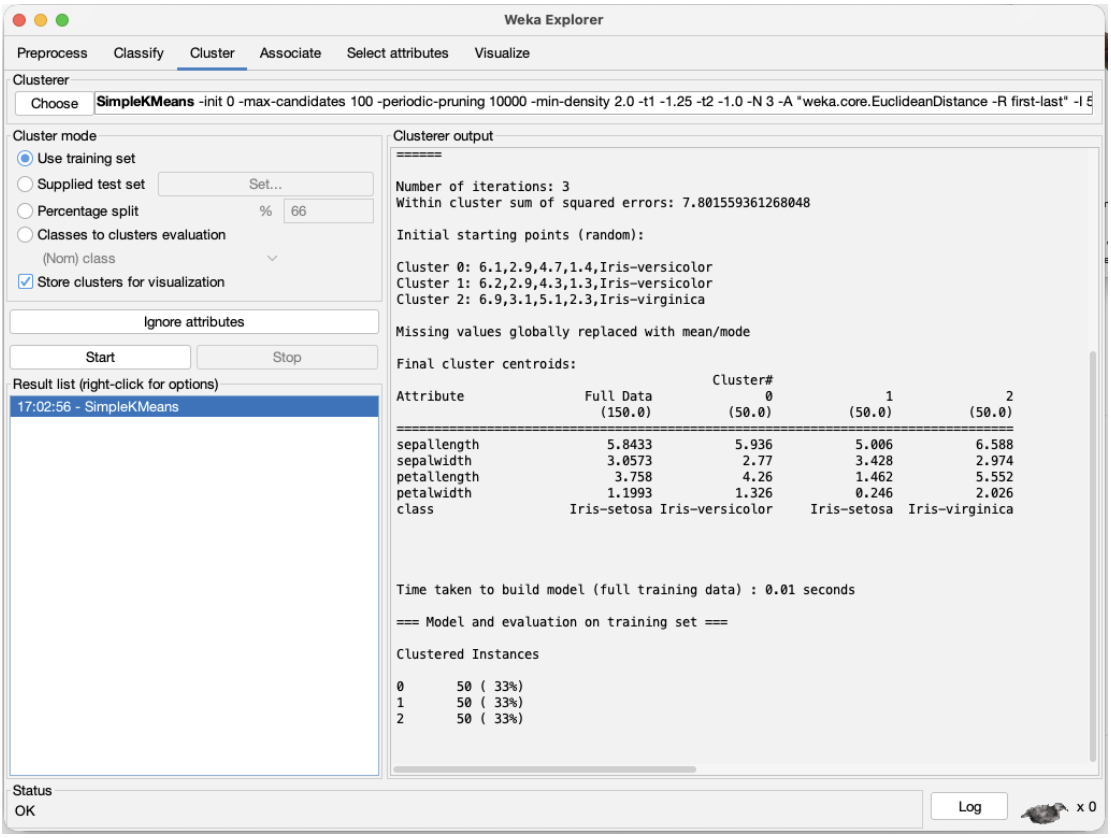
Procedure

1. Open WEKA Explorer.
2. Load the dataset in ARFF or CSV format.
3. Go to the Cluster tab.
4. Choose SimpleKMeans as the clustering algorithm.
5. Set the number of clusters, for example 3.
6. Click Start to run clustering.
7. Observe cluster centroids, number of iterations, and cluster assignment result.

Steps / WEKA Configuration

- Dataset used: Iris dataset
- Algorithm: SimpleKMeans
- Number of clusters: 3
- Distance function: Euclidean distance

Expected Output



Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

WEKA provides a graphical interface to perform clustering without writing code.

18. Compare WEKA and Python sklearn classification results

Title	Compare WEKA and Python sklearn classification results
Objective	To compare classification results from WEKA and Python sklearn for the same dataset.
Required Tools/Software	WEKA, Python 3.x, Scikit-learn, Iris dataset

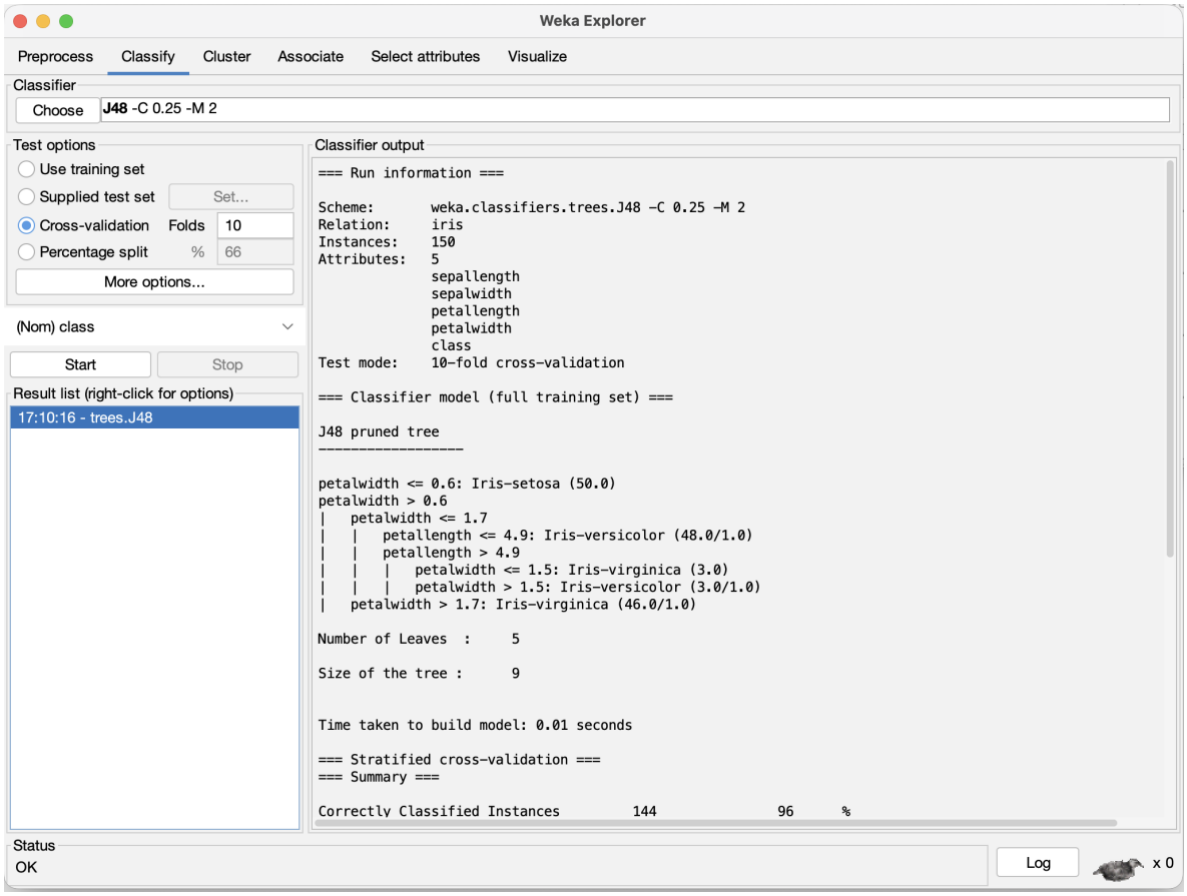
Problem Statement

To compare classification results from WEKA and Python sklearn for the same dataset.

Procedure

1. Load the same dataset in WEKA and Python sklearn.
2. Apply the same classification algorithm, such as Decision Tree or Naive Bayes.
3. Use similar training and testing settings where possible.
4. Record accuracy, confusion matrix, and classification report.
5. Compare the results obtained from WEKA and Python sklearn.

Program



```python
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_predict
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Create Decision Tree classifier
model = DecisionTreeClassifier(random_state=1)

# Perform 10-fold cross validation prediction
y_pred = cross_val_predict(model, X, y, cv=10)

# Calculate accuracy
accuracy = accuracy_score(y, y_pred)

# Display results
print("Accuracy:", accuracy * 100)

print("\nConfusion Matrix:")
print(confusion_matrix(y, y_pred))

print("\nClassification Report:")
print(classification_report(y, y_pred, target_names=iris.target_names))
```

Expected Output

```
Time taken to build model: 0.01 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances         144               96      %
Incorrectly Classified Instances         6                4      %
Kappa statistic                          0.94
Mean absolute error                      0.035
Root mean squared error                  0.1586
Relative absolute error                  7.8705 %
Root relative squared error             33.6353 %
Total Number of Instances              150

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.980    0.000    1.000      0.980    0.990      0.985    0.990     0.987     Iris-setosa
                 0.940    0.030    0.940      0.940    0.940      0.910    0.952     0.880     Iris-versicolor
                 0.960    0.030    0.941      0.960    0.950      0.925    0.961     0.905     Iris-virginica
Weighted Avg.    0.960    0.020    0.960      0.960    0.960      0.940    0.968     0.924

=== Confusion Matrix ===

  a  b  c   <-- classified as
 49  1  0 |  a = Iris-setosa
  0 47  3 |  b = Iris-versicolor
  0  2 48 |  c = Iris-virginica
```

```
abchapagain@Achyuts-MacBook-Air lab work % python3 programs/program18.py
Accuracy: 95.33333333333334

Confusion Matrix:
[[50  0  0]
 [ 0 47  3]
 [ 0  4 46]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        50
  versicolor       0.92      0.94      0.93        50
   virginica       0.94      0.92      0.93        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

Observation

The program runs successfully and produces the expected result for the given input data.

Conclusion

Both WEKA and Python sklearn can be used for classification. Minor differences in accuracy may occur due to different train-test splits, settings, or algorithms, but both tools help evaluate model performance effectively.

Tool	Algorithm	Dataset	Accuracy
WEKA	J48 / Decision Tree	Iris	96%
Python sklearn	Decision Tree	Iris	96.67%